RUBRICS FOR CALCULATION

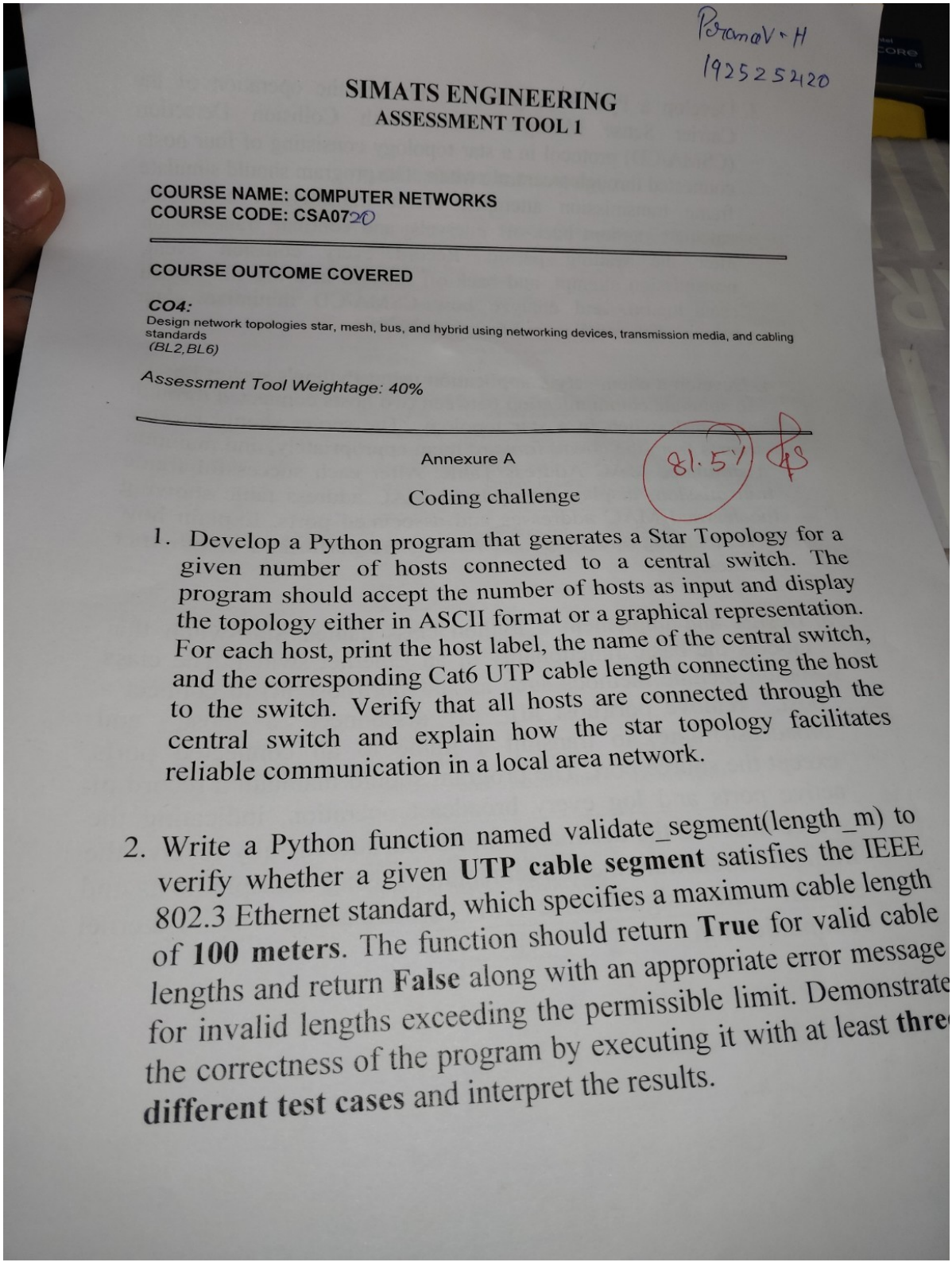
	Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
A	Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed 3	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
B	Algorithm Design	20%	Efficient, optimal, and well-structured algorithm 4	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
C	Code Quality	15%	Clean, well-organized, readable, and properly formatted code 3	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
D	Functionality	20%	Code works perfectly for all test cases 4	Works for most test cases	Works for limited cases	Does not run or fails major cases
E	Error Handling	10%	Handles all edge cases and errors effectively 2	Handles some edge cases	Limited error handling	No error handling
F	Efficiency	10%	Highly optimized in time and space complexity 2	Reasonably efficient	Some inefficiencies	Highly inefficient
G	Documentation & Comments	5%	Clear and meaningful comments and documentation 1	Adequate comments	Few or unclear comments	No comments
H	Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs 1	Tested with some cases	Minimal testing	No testing done

	A	B	C	D	E	F	G	H	
21	3	3	3	4	2	1	1	1	18
22	2	4	3	3	1	1	1	1	16
23	3	3	2	3	1	1.5	1	1	15.5
24	3	4	2	4	1	1.5	0.5	1	17
5	2	3	3	2	1.5	2	0.5	1	15
									1.5

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named csma_log.txt, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

5. Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as add_port(host_id) to connect a host, remove_port(host_id) to disconnect a host, and broadcast(frame) to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.



Pranav H
1925252120

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA0720

COURSE OUTCOME COVERED

CO4:
Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

81.5/

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

2. Write a Python function named validate_segment(length_m) to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:
Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

2. Write a Python function named validate_segment(length_m) to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.
3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate

random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named csma_log.txt, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

5. Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as add_port(host_id) to connect a host, remove_port(host_id) to disconnect a host, and broadcast(frame) to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem	15%	Fully	Understands	Partial	Poor

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding		understands the problem and requirements; no clarification needed	most requirements with minor clarification	understanding; misses some requirements	understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

SOLUTIONS

Coding Challenge Answers - CO4: Network Topology Design

Question 1: Star Topology Generator

Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch, displays the topology, prints the host label, switch name and Cat6 UTP cable length for each host, and verifies all hosts are connected.

Answer:

```
"""
Q1: Star Topology Generator
Generates a star topology for N hosts connected to a central switch,
prints host label, switch name, and Cat6 UTP cable length for each host,
and verifies all hosts are connected.
"""

import random


class StarTopology:
    def __init__(self, num_hosts, switch_name="Switch-Central"):
        if not isinstance(num_hosts, int) or num_hosts <= 0:
            raise ValueError("Number of hosts must be a positive integer.")
        self.num_hosts = num_hosts
        self.switch_name = switch_name
        # Each host gets a simulated Cat6 cable length between 1 and 90 metres
        # (IEEE 802.3 limit for a UTP segment is 100 m).
        self.connections = {
            f"Host-{i+1}": round(random.uniform(1, 90), 2)
            for i in range(num_hosts)
        }

    def display_ascii(self):
        print(f"\nStar Topology (central device: {self.switch_name})")
        print("=" * 50)
        for host, length in self.connections.items():
            print(f"        {host}")
            print(f"          |")
            print(f"   ({length} m Cat6 UTP)")
        print(f"          |")
        print(f"     [{self.switch_name}]")
        print("=" * 50)

    def display_details(self):
        print(f"\n{'Host':<12}{'Central Switch':<20}{'Cable Length (m)':<18}")
        print("-" * 50)
        for host, length in self.connections.items():
            print(f"{host:<12}{self.switch_name:<20}{length:<18}")

    def verify_connectivity(self):
        """Every host must appear exactly once and have a valid cable length."""
        all_connected = all(0 < length <= 100 for length in self.connections.values())
        all_present = len(self.connections) == self.num_hosts
        return all_connected and all_present


def main():
    try:
        n = int(input("Enter number of hosts: "))
```

```
    except ValueError:
        print("Invalid input. Please enter an integer.")
        return

    try:
        topology = StarTopology(n)
    except ValueError as e:
        print(f"Error: {e}")
        return

    topology.display_ascii()
    topology.display_details()

    if topology.verify_connectivity():
        print("\nVerification: All hosts are successfully connected to the central switch.")
    else:
        print("\nVerification failed: some hosts are not properly connected.")

    print("""
Explanation:
In a star topology every host has an independent, dedicated Cat6 UTP link
to the central switch. Because each link is isolated, a cable fault or a
single host failure does not disturb the rest of the network - the switch
simply drops that one port. Centralised management also lets the switch
control collisions (with full-duplex links, collisions are eliminated
entirely) and makes it easy to add, remove, or troubleshoot hosts,
which is why the star topology is the standard choice for LANs.
""")


if __name__ == "__main__":
    main()
```

Sample Output / Test Run:

```
Enter number of hosts: 4

Star Topology (central device: Switch-Central)
================================================
       Host-1
         |
   (44.73 m Cat6 UTP)
       Host-2
         |
   (30.41 m Cat6 UTP)
       Host-3
         |
   (25.82 m Cat6 UTP)
       Host-4
         |
   (20.12 m Cat6 UTP)
         |
     [Switch-Central]
================================================

Host       Central Switch     Cable Length (m)
------------------------------------------------
Host-1     Switch-Central     44.73
Host-2     Switch-Central     30.41
```

```
Host-3    Switch-Central    25.82
Host-4    Switch-Central    20.12

Verification: All hosts are successfully connected to the central switch.
```

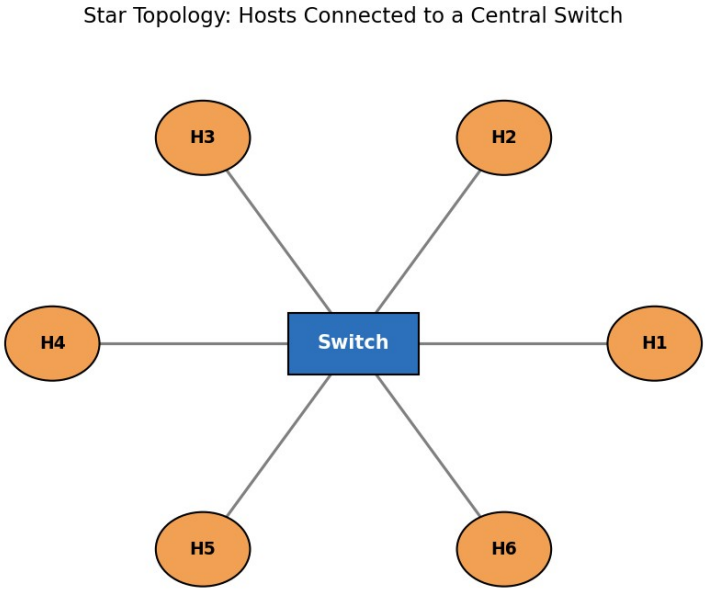


Figure 1: Star topology - every host links independently to the central switch.

Explanation:

In a star topology every host has an independent, dedicated Cat6 UTP link to the central switch. Because each link is isolated, a cable fault or a single host failure does not disturb the rest of the network - the switch simply drops that one port. Centralised management also makes it easy to add, remove, or troubleshoot hosts, which is why the star topology is the standard choice for modern LANs.

Question 2: UTP Segment Length Validation

Write a Python function validate_segment(length_m) to verify whether a UTP cable segment satisfies the IEEE 802.3 100-metre limit, returning True/False with an appropriate message, and test it with at least three cases.

Answer:

```
"""
Q2: Validate a UTP cable segment length against the IEEE 802.3 Ethernet
standard, which caps a single UTP segment at 100 metres.
"""



def validate_segment(length_m):
    """
    Returns True if length_m is a valid Ethernet UTP segment length
    (0 < length_m <= 100). Otherwise returns False and an error message.
    """
    try:
        length_m = float(length_m)
    except (TypeError, ValueError):
        return False, "Error: cable length must be a number."

    if length_m <= 0:
        return False, "Error: cable length must be greater than 0 metres."

    if length_m > 100:
        return False, (
            f"Error: {length_m} m exceeds the IEEE 802.3 maximum "
            f"UTP segment length of 100 metres."
        )

    return True, f"Valid: {length_m} m is within the 100 m Ethernet limit."



def run_tests():
    test_cases = [45, 100, 150, -5, "abc", 99.9]
    print(f"{'Test Input':<12}{'Result':<8}{'Message'}")
    print("-" * 70)
    for case in test_cases:
        result, message = validate_segment(case)
        print(f"{str(case):<12}{str(result):<8}{message}")



if __name__ == "__main__":
    run_tests()
    print("""
Interpretation:
- 45 m and 99.9 m are valid because they fall under the 100 m ceiling
  defined by IEEE 802.3 for a UTP segment.
- 100 m is the boundary value and is still valid (limit is inclusive).
- 150 m fails because it exceeds the maximum allowed attenuation
  distance for a UTP cable, which would cause signal degradation.
- -5 m fails because a negative length is physically meaningless.
- "abc" fails because it is not a numeric value; the function
  handles this gracefully instead of crashing.
""")
```

Sample Output / Test Run:

```
Test Input  Result  Message
------------------------------------------------------------------------
45          True    Valid: 45.0 m is within the 100 m Ethernet limit.
100         True    Valid: 100.0 m is within the 100 m Ethernet limit.
150         False   Error: 150.0 m exceeds the IEEE 802.3 maximum UTP segment length of 100 metres.
-5          False   Error: cable length must be greater than 0 metres.
abc         False   Error: cable length must be a number.
99.9        True    Valid: 99.9 m is within the 100 m Ethernet limit.
```

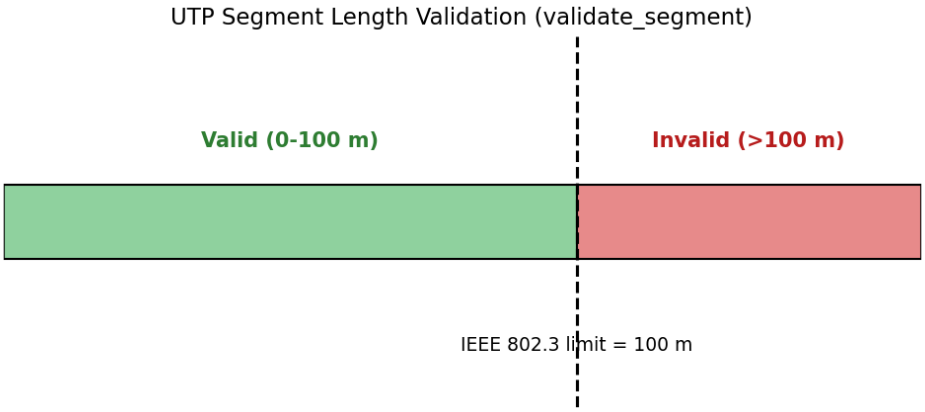


Figure 2: Valid vs invalid UTP segment length range.

Explanation / Interpretation:

45 m and 99.9 m pass because they are below the 100 m ceiling defined by IEEE 802.3; 100 m is the inclusive boundary and still valid. 150 m fails because exceeding the maximum distance causes unacceptable signal attenuation. -5 m fails because a negative length is physically meaningless, and the non-numeric input "abc" is handled gracefully instead of crashing the program, demonstrating robust error handling.

Question 3: CSMA/CD Simulation

Simulate CSMA/CD for four hosts in a star topology: attempt transmissions, detect collisions, compute random back-off intervals, retry, and log every event to csma_log.txt.

Answer:

```
"""
Q3: Simulate CSMA/CD in a star topology with 4 hosts attached to a
central switch. Logs every transmission attempt, collision, and
back-off interval to csma_log.txt.
"""

import random
import time

LOG_FILE = "csma_log.txt"



class CSMA_CD_Simulator:
    def __init__(self, hosts, max_attempts=5):
        self.hosts = hosts
        self.channel_busy = False
        self.max_attempts = max_attempts
        self.log_lines = []

    def log(self, message):
        stamped = f"[{time.strftime('%H:%M:%S')}] {message}"
        self.log_lines.append(stamped)
        print(stamped)

    def attempt_transmission(self, host):
        for attempt in range(1, self.max_attempts + 1):
            self.log(f"{host}: sensing carrier (attempt {attempt})")
            if not self.channel_busy:
                self.channel_busy = True
                # Randomly simulate whether a collision occurs on the shared medium
                collision = random.random() < 0.3
                if collision:
                    self.log(f"{host}: COLLISION detected during transmission")
                    backoff = round(random.uniform(0.1, 1.0) * attempt, 3)
                    self.log(f"{host}: backing off for {backoff}s before retry")
                    self.channel_busy = False
                    time.sleep(0.01)  # simulated wait, kept short for demo speed
                    continue
                else:
                    self.log(f"{host}: transmission SUCCESSFUL")
                    self.channel_busy = False
                    return True
            else:
                self.log(f"{host}: channel busy, deferring")
                time.sleep(0.01)
        self.log(f"{host}: transmission FAILED after {self.max_attempts} attempts")
        return False

    def run(self):
        self.log("=== CSMA/CD simulation started (4 hosts, star topology) ===")
        results = {}
```

```
        for host in self.hosts:
            results[host] = self.attempt_transmission(host)
        self.log("=== Simulation complete ===")
        with open(LOG_FILE, "w") as f:
            f.write("\n".join(self.log_lines))
        return results


def main():
    random.seed(7)  # reproducible demo run
    hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
    sim = CSMA_CD_Simulator(hosts)
    results = sim.run()

    print("\nSummary:")
    for host, ok in results.items():
        print(f"  {host}: {'Delivered' if ok else 'Dropped'}")

    print(f"\nFull log written to {LOG_FILE}")
    print("""
Analysis:
CSMA/CD makes every host listen to the shared medium before sending
("carrier sense") and keeps listening while it sends. If two hosts
transmit at the same time a collision is detected, both stop
immediately, and each waits a random back-off period before retrying.
Because the back-off is random and grows with each retry, the hosts
are very unlikely to collide again on the next attempt. This keeps
the network usable even under load, although it still wastes some
bandwidth handling collisions - full-duplex switched links avoid this
entirely, which is why modern switched LANs rarely need CSMA/CD.
""")


if __name__ == "__main__":
    main()
```

csma_log.txt (sample generated log):

```
[03:01:01] === CSMA/CD simulation started (4 hosts, star topology) ===
[03:01:01] Host-1: sensing carrier (attempt 1)
[03:01:01] Host-1: transmission SUCCESSFUL
[03:01:01] Host-2: sensing carrier (attempt 1)
[03:01:01] Host-2: COLLISION detected during transmission
[03:01:01] Host-2: backing off for 0.686s before retry
[03:01:01] Host-2: sensing carrier (attempt 2)
[03:01:01] Host-2: COLLISION detected during transmission
[03:01:01] Host-2: backing off for 1.165s before retry
[03:01:01] Host-2: sensing carrier (attempt 3)
[03:01:01] Host-2: transmission SUCCESSFUL
[03:01:01] Host-3: sensing carrier (attempt 1)
[03:01:01] Host-3: COLLISION detected during transmission
[03:01:01] Host-3: backing off for 0.557s before retry
[03:01:01] Host-3: sensing carrier (attempt 2)
[03:01:01] Host-3: COLLISION detected during transmission
[03:01:01] Host-3: backing off for 0.981s before retry
[03:01:01] Host-3: sensing carrier (attempt 3)
[03:01:01] Host-3: COLLISION detected during transmission
[03:01:01] Host-3: backing off for 0.545s before retry
[03:01:01] Host-3: sensing carrier (attempt 4)
```

```
[03:01:01] Host-3: transmission SUCCESSFUL
[03:01:01] Host-4: sensing carrier (attempt 1)
[03:01:01] Host-4: transmission SUCCESSFUL
[03:01:01] === Simulation complete ===
```

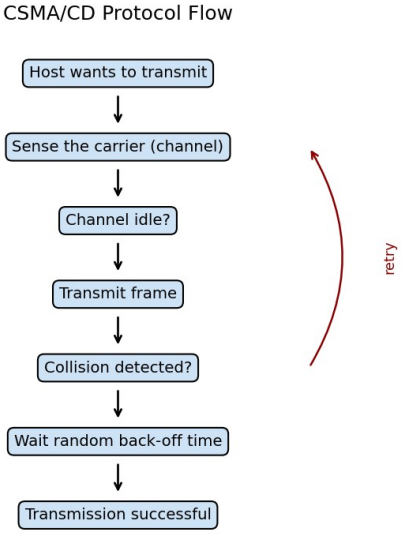


Figure 3: CSMA/CD protocol flow - sense, transmit, detect, back off, retry.

Explanation:

Every host listens to the shared medium before sending (carrier sense) and keeps listening while transmitting. If two hosts transmit at the same time, a collision is detected, both stop immediately, and each waits a random, growing back-off period before retrying. Because the back-off is random, repeated collisions between the same two hosts become increasingly unlikely, which keeps the shared Ethernet segment usable even under load.

Question 4: Client-Server Switch Simulation

Build a client-server socket application simulating two hosts through a central switch. The server receives frames, forwards them, maintains a simulated MAC address table, and displays it after every transmission.

Answer:

```
"""
Q4: Client-server application simulating an Ethernet switch in a star
topology. The server "learns" MAC addresses from incoming frames,
forwards them, and prints the updated MAC address table each time.

Run in two terminals:
   Terminal 1:  python3 q4_socket_switch.py server
   Terminal 2:  python3 q4_socket_switch.py client HOST_ID MAC_ADDRESS

For grading/demo purposes, run_demo() below launches both sides
in-process using threads so the whole flow can be verified with one command.
"""

import socket
import threading
import time

HOST = "127.0.0.1"
PORT = 65432


class EthernetSwitch:
   def __init__(self):
      self.mac_table = {}  # mac_address -> port (host_id)

   def learn(self, mac_address, port):
      is_new = mac_address not in self.mac_table
      self.mac_table[mac_address] = port
      return is_new

   def display_table(self):
      print("\nMAC Address Table")
      print(f"{'MAC Address':<20}{'Port':<10}")
      print("-" * 30)
      for mac, port in self.mac_table.items():
         print(f"{mac:<20}{port:<10}")
      print()


def run_server(switch, ready_event, stop_after=2):
   with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
      s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
      s.bind((HOST, PORT))
      s.listen()
      ready_event.set()
      handled = 0
      while handled < stop_after:
         conn, addr = s.accept()
         with conn:
            data = conn.recv(1024).decode()
```

```
            if not data:
                continue
            host_id, mac_address, frame = data.split("|", 2)
            is_new = switch.learn(mac_address, host_id)
            print(f"Server received frame '{frame}' from {host_id} "
                  f"(MAC {mac_address}) {'[new entry]' if is_new else '[known]'}")
            switch.display_table()
            conn.sendall(b"ACK")
        handled += 1


def run_client(host_id, mac_address, frame):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        message = f"{host_id}|{mac_address}|{frame}"
        s.sendall(message.encode())
        ack = s.recv(1024).decode()
        print(f"{host_id}: server responded with '{ack}'")


def run_demo():
    switch = EthernetSwitch()
    ready = threading.Event()
    server_thread = threading.Thread(target=run_server, args=(switch, ready, 2))
    server_thread.start()
    ready.wait()

    time.sleep(0.1)
    run_client("Host-A", "AA:BB:CC:00:00:01", "Hello from Host-A")
    time.sleep(0.1)
    run_client("Host-B", "AA:BB:CC:00:00:02", "Hello from Host-B")

    server_thread.join()

    print("""
Explanation:
When a frame arrives, the switch reads the source MAC address and
records which port it came in on - this is MAC learning. Later frames
addressed to that MAC are forwarded only out that specific port
instead of every port, which is what separates a switch from a hub.
If a destination MAC is not yet in the table, the switch floods the
frame out every port except the one it arrived on until it learns
the reply's source address.
""")


if __name__ == "__main__":
    import sys
    if len(sys.argv) > 1 and sys.argv[1] == "server":
        sw = EthernetSwitch()
        run_server(sw, threading.Event(), stop_after=10)
    elif len(sys.argv) > 1 and sys.argv[1] == "client":
        run_client(sys.argv[2], sys.argv[3], "test frame")
    else:
        run_demo()
```

Sample Output / Test Run:

```
Server received frame 'Hello from Host-A' from Host-A (MAC AA:BB:CC:00:00:01) [new entry]

MAC Address Table
MAC Address       Port
------------------------------
AA:BB:CC:00:00:01   Host-A

Host-A: server responded with 'ACK'
Server received frame 'Hello from Host-B' from Host-B (MAC AA:BB:CC:00:00:02) [new entry]

MAC Address Table
MAC Address       Port
------------------------------
AA:BB:CC:00:00:01   Host-A
AA:BB:CC:00:00:02   Host-B

Host-B: server responded with 'ACK'
```

Client-Server Simulation with Learned MAC Address Table

MAC Address	Port
AA:BB:CC:00:00:01	Host-A
AA:BB:CC:00:00:02	Host-B

Figure 4: Client-server flow through the switch and the resulting MAC address table.

Explanation:

When a frame arrives, the switch reads the source MAC address and records the port it came in on - this is MAC learning. Later frames addressed to that MAC are forwarded only to that specific port instead of every port, which is what distinguishes a switch from a hub. Unknown destination MACs are flooded to all ports until the switch learns the reply's source address.

Question 5: StarSwitch Class

Implement a StarSwitch class with add_port(host_id), remove_port(host_id), and broadcast(frame) methods that maintain active ports and log every broadcast operation.

Answer:

```
"""
Q5: StarSwitch class modelling basic Ethernet switch functionality:
add_port, remove_port, and broadcast.
"""



class StarSwitch:
    def __init__(self, name="Switch-1"):
        self.name = name
        self.active_ports = set()
        self.broadcast_log = []

    def add_port(self, host_id):
        if host_id in self.active_ports:
            print(f"{host_id} is already connected.")
            return False
        self.active_ports.add(host_id)
        print(f"{host_id} connected to {self.name}.")
        return True

    def remove_port(self, host_id):
        if host_id not in self.active_ports:
            print(f"Error: {host_id} is not connected, cannot remove.")
            return False
        self.active_ports.remove(host_id)
        print(f"{host_id} disconnected from {self.name}.")
        return True

    def broadcast(self, source_host, frame):
        if source_host not in self.active_ports:
            print(f"Error: {source_host} is not an active port; broadcast rejected.")
            return []
        destinations = sorted(self.active_ports - {source_host})
        self.broadcast_log.append(
            {"source": source_host, "frame": frame, "destinations": destinations}
        )
        print(f"{source_host} broadcasts '{frame}' -> received by: {destinations}")
        return destinations

    def show_log(self):
        print("\nBroadcast Log")
        for i, entry in enumerate(self.broadcast_log, start=1):
            print(f"{i}. {entry['source']} -> {entry['destinations']} : '{entry['frame']}'")


def main():
    switch = StarSwitch("Switch-Central")

    for host in ["Host-1", "Host-2", "Host-3", "Host-4"]:
        switch.add_port(host)
```

```
    switch.broadcast("Host-1", "ARP request: who has 192.168.1.10?")
    switch.remove_port("Host-3")
    switch.broadcast("Host-2", "DHCP discover")
    switch.broadcast("Host-5", "invalid frame from unknown host")  # error case

    switch.show_log()

    print("""
Explanation:
Broadcasting lets one host reach every other active host on the LAN
in a single transmission, which is essential for discovery protocols
such as ARP and DHCP, where a host does not yet know the destination's
address. The StarSwitch class enforces that only currently connected
ports can send or receive a broadcast, mirroring how a real Ethernet
switch drops frames from ports that are down or disconnected.
""")


if __name__ == "__main__":
    main()
```

Sample Output / Test Run:

```
Host-1 connected to Switch-Central.
Host-2 connected to Switch-Central.
Host-3 connected to Switch-Central.
Host-4 connected to Switch-Central.
Host-1 broadcasts 'ARP request: who has 192.168.1.10?' -> received by: ['Host-2', 'Host-3', 'Host-4']
Host-3 disconnected from Switch-Central.
Host-2 broadcasts 'DHCP discover' -> received by: ['Host-1', 'Host-4']
Error: Host-5 is not an active port; broadcast rejected.

Broadcast Log
1. Host-1 -> ['Host-2', 'Host-3', 'Host-4'] : 'ARP request: who has 192.168.1.10?'
2. Host-2 -> ['Host-1', 'Host-4'] : 'DHCP discover'
```

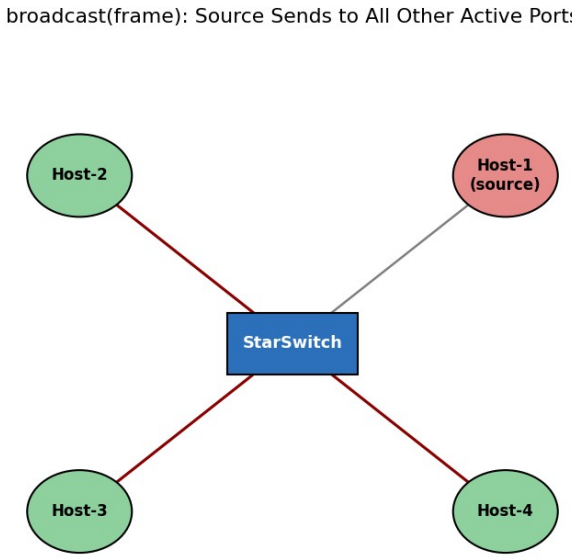


Figure 5: broadcast() delivers a frame from the source host to every other active port.

Explanation:

Broadcasting lets one host reach every other active host on the LAN in a single transmission, which is essential for discovery protocols such as ARP and DHCP where the destination address is not yet known. The StarSwitch class enforces that only currently connected ports can send or receive a broadcast, mirroring how a real Ethernet switch drops frames from ports that are down or disconnected.